



INSTITUTO POLITÉCNICO NACIONAL
Unidad Interdisciplinaria de Ingeniería
ciencias sociales y
Administrativas



ABSTRACCIÓN Y
USO DE DATOS

Secuencia: 3CM30

Integrantes:

Gomez Rodriguez Aldo Sebastian - NL13

Nieto Ramirez Jean - NL 31

Nuñez Solis Luis Angel - NL 32

Link de Repositorio:

<https://github.com/LuisxD14/Abstraccion>

Ordenamiento Burbuja

Concepto general

El presente programa implementa el algoritmo de ordenamiento burbuja (Bubble Sort)

El objetivo principal es ordenar diferentes tipos de datos:

- Números enteros
- Caracteres
- Un tipo de dato personalizado (estructura **Estudiante**)

Se hace uso de polimorfismo y herencia, permitiendo reutilizar la misma lógica de ordenamiento adaptada a distintos tipos de datos.

Descripción del Programa

El sistema está compuesto por:

1. Clase abstracta **Ordenador**

Define la estructura base del programa mediante métodos virtuales puros:

- **cargar()** → Captura datos del usuario
- **ordenar()** → Aplica el algoritmo burbuja
- **mostrar()** → Imprime los resultados

Esto obliga a las clases derivadas a implementar su propia versión de estos métodos.

2. Clases Derivadas

OrdenadorInt

- Maneja un arreglo de enteros
- Ordena de menor a mayor

OrdenadorChar

- Maneja caracteres
- Ordena alfabéticamente

OrdenadorEstudiante

- Maneja una estructura personalizada:

```
struct Estudiante {  
    string nombre;  
    float promedio;  
};
```

```
};
```

- Ordena a los estudiantes por promedio (de menor a mayor)

Algoritmo Utilizado: Burbuja

El método de ordenamiento burbuja compara elementos adyacentes y los intercambia si están en el orden incorrecto.

Características:

- Fácil de implementar
- Poco eficiente para grandes volúmenes de datos

Complejidad:

- Peor caso: $O(n^2)$
- Mejor caso: $O(n)$

Funcionamiento del Programa

Flujo general:

1. Se muestra un menú al usuario
2. El usuario selecciona el tipo de dato a ordenar
3. Se crea un objeto de la clase correspondiente
4. Se ejecutan los métodos:
 - `cargar()`
 - `ordenar()`
 - `mostrar()`
5. El proceso se repite hasta que el usuario decide salir

Herencia

Las clases `OrdenadorInt`, `OrdenadorChar` y `OrdenadorEstudiante` heredan de la clase base `Ordenador`.

Polimorfismo

Se utiliza un puntero de tipo `Ordenador*` para ejecutar distintos comportamientos dependiendo del objeto:

```
void ejecutar(Ordenador* o) {  
    o->cargar();  
    o->ordenar();  
    o->mostrar();  
}
```

Abstracción

La clase base define la interfaz general sin mostrar la implementación específica de cada tipo de dato.

Ejemplo de Ejecución

Entrada:

Enteros: 5 2 9 1 3

Salida:

Resultado: 1 2 3 5 9

Ordenamiento Merge Sort en C++

Concepto General

El presente programa implementa el algoritmo de ordenamiento **Merge Sort**. Este algoritmo pertenece a la categoría de métodos de ordenamiento eficientes y se basa en la técnica de **divide y vencerás**.

El objetivo del programa es ordenar un conjunto de datos (en este caso números enteros), organizándolos de menor a mayor mediante un proceso recursivo de división y combinación.

Descripción del Algoritmo Merge Sort

Merge Sort funciona dividiendo el arreglo en partes más pequeñas hasta que cada subarreglo contiene un solo elemento. Posteriormente, estos subarreglos se combinan de forma ordenada hasta reconstruir el arreglo completo.

$$T(n) = 2T(n/2) + n$$

Funcionamiento general:

1. Dividir el arreglo en dos mitades
2. Aplicar recursivamente Merge Sort a cada mitad
3. Combinar (merge) las mitades ordenadas
4. Obtener el arreglo final ordenado

Descripción del Código

El programa está compuesto por tres funciones principales:

1. Función **merge**

Se encarga de combinar dos subarreglos previamente ordenados en uno solo.

```
void merge(int arr[], int izq, int mid, int der)
```

Proceso:

- Se crean dos arreglos auxiliares (**L** y **R**)
- Se copian los datos del arreglo original
- Se comparan los elementos de ambos arreglos
- Se insertan en orden en el arreglo original

2. Función **mergeSort**

Implementa la recursividad del algoritmo.

```
void mergeSort(int arr[], int izq, int der)
```

Proceso:

- Verifica si el arreglo puede dividirse
- Calcula el punto medio
- Llama recursivamente a sí misma
- Finalmente llama a **merge** para unir los resultados

3. Función **mostrar**

Permite imprimir el arreglo ordenado.

```
void mostrar(int arr[], int n)
```

Funcionamiento del Programa

Flujo general:

1. Se define un arreglo de enteros
2. Se llama a la función **mergeSort**
3. El arreglo se divide recursivamente
4. Se combinan los subarreglos ordenados
5. Se muestra el resultado final

Características del Algoritmo

- Tipo: Ordenamiento por comparación
- Técnica: Divide y vencerás
- Estable: Sí
- Uso de memoria: Requiere memoria adicional

Complejidad

- Mejor caso: $O(n \log n)$
- Caso promedio: $O(n \log n)$
- Peor caso: $O(n \log n)$

Esto lo hace más eficiente que algoritmos simples como burbuja o selección.

Ejemplo de Ejecución

Entrada:

5 2 9 1 3

Salida:

1 2 3 5 9

ordenamiento Quick Sort

Concepto General

El presente programa implementa el algoritmo de ordenamiento **Quick Sort**. Este algoritmo es uno de los métodos de ordenamiento más eficientes y utilizados, basado en la técnica de **divide y vencerás**.

El objetivo del programa es ordenar un conjunto de datos compuesto por un tipo de dato personalizado (**Dato**), el cual incluye:

- Un carácter
- Un número entero

El ordenamiento puede realizarse en función de cualquiera de estos dos atributos, según la elección del usuario.

Descripción del Tipo de Dato

El programa utiliza una estructura personalizada:

```
struct Dato {  
    char letra;  
    int numero;  
};
```

Esta estructura permite almacenar información compuesta, lo que hace posible

ordenar por diferentes criterios.

Descripción del Algoritmo Quick Sort

Quick Sort selecciona un elemento como pivote y reorganiza el arreglo de manera que:

- Los elementos menores al pivote queden a la izquierda
- Los elementos mayores queden a la derecha

$$T(n)=T(k)+T(n-k-1)+n$$

Funcionamiento general:

1. Seleccionar un pivote (en este caso, el último elemento)
2. Reordenar el arreglo en torno al pivote
3. Aplicar el proceso recursivamente a las sublistas izquierda y derecha
4. Obtener el arreglo ordenado

Descripción del Código

El programa está compuesto por las siguientes funciones:

1. Función **intercambiar**

```
void intercambiar(Dato &a, Dato &b)
```

Permite intercambiar dos elementos de tipo **Dato**.

2. Función **particion**

```
int particion(Dato arreglo[], int bajo, int alto, int criterio)
```

Función principal del algoritmo:

- Selecciona el pivote
- Reorganiza los elementos según el criterio elegido

Criterio:

- **1** → Ordena por carácter
- **2** → Ordena por número

3. Función **ejecutarQuickSort**

```
void ejecutarQuickSort(Dato arreglo[], int bajo, int alto, int criterio)
```

Proceso:

- Aplica recursividad

- Divide el arreglo en subproblemas
- Llama a **particion** para ordenar

4. Función **mostrarLista**

void mostrarLista(Dato arreglo[], int n)

Imprime los elementos del arreglo en formato:

[letra, numero]

Funcionamiento del Programa

Flujo general:

1. El usuario ingresa la cantidad de elementos
2. Se capturan los datos (carácter y número)
3. El usuario elige el criterio de ordenamiento
4. Se ejecuta Quick Sort
5. Se muestran los resultados ordenados

Características del Algoritmo

- Tipo: Ordenamiento por comparación
- Técnica: Divide y vencerás
- En el lugar (in-place): Sí
- Estable: No (puede cambiar el orden relativo)

Complejidad

- Mejor caso: $O(n \log n)$
- Caso promedio: $O(n \log n)$
- Peor caso: $O(n^2)$ (cuando el pivote es el menor o mayor elemento)

Ejemplo de Ejecución

Entrada:

[a, 5] [c, 2] [b, 8]

Criterio: 1 (por carácter)

Salida:

[a, 5] [b, 8] [c, 2]

Implementación de una Pila mediante Arreglo

Concepto General

El presente programa implementa una **estructura de datos tipo pila (stack)** utilizando un arreglo estático

Una pila es una estructura lineal que sigue el principio **LIFO (Last In, First Out)**, es decir, el último elemento en entrar es el primero en salir.

En este caso, la pila se implementa mediante un arreglo de tamaño fijo, lo que implica que su capacidad está limitada.

Descripción de la Estructura

La pila está definida mediante una clase **Pila**, que contiene:

- Un arreglo de enteros (**datos**)
- Un contador (**cantidad**) que indica el número de elementos almacenados

El tamaño máximo de la pila es de **100 elementos**.

Operaciones Implementadas

1. Método **agregar(int valor)**

Permite insertar un elemento en la pila.

- Inserta el valor en la posición actual
- Incrementa el contador

Condición:

- Solo se agrega si la pila no está llena

2. Método **quitar()**

Elimina el elemento en la parte superior de la pila.

- Disminuye el contador
- No elimina físicamente el dato, solo reduce el acceso

Condición:

- Solo se ejecuta si la pila no está vacía

3. Método **mostrar()**

Imprime los elementos de la pila en formato vertical:

| elemento |

Se muestran desde el tope hacia la base.

4. Método **estaVacía()**

Verifica si la pila no contiene elementos.

Retorna:

- **true** si está vacía
- **false** en caso contrario

5. Método **estaLlena()**

Verifica si la pila alcanzó su capacidad máxima.

Retorna:

- **true** si está llena
- **false** en caso contrario

6. Método **obtenerTamano()**

Devuelve la cantidad actual de elementos almacenados en la pila.

Funcionamiento del Programa

Flujo general:

1. Se crea un objeto de tipo **Pila**
2. Se muestra un menú interactivo
3. El usuario selecciona una opción
4. Se ejecuta la operación correspondiente
5. El proceso se repite hasta que el usuario decide salir

Menú del Sistema

El programa ofrece las siguientes opciones:

1. Agregar elemento
2. Quitar elemento
3. Mostrar pila
4. Verificar si está vacía
5. Verificar si está llena
6. Mostrar tamaño de la pila
7. Salir

Características de la Implementación

- Tipo de estructura: Lineal
- Implementación: Arreglo estático
- Capacidad: Fija (100 elementos)
- Acceso: Restringido al tope de la pila

Implementación de una Pila Estática con Tipo de Dato Nuevo y arreglos

Concepto General

El presente programa implementa una **estructura de datos tipo pila (stack)** utilizando un arreglo estático en C++.

Una pila es una estructura lineal que opera bajo el principio **LIFO (Last In, First Out)**, lo que significa que el último elemento en ingresar es el primero en salir.

En este caso, la pila no almacena datos simples, sino un **tipo de dato personalizado (Persona)**, lo que permite manejar información más compleja.

Descripción del Tipo de Dato

El programa utiliza una estructura **Persona**, la cual contiene:

```
struct Persona {  
  
    char nombre[50];  
  
    int edad;  
  
};
```

Esta estructura permite almacenar:

- Nombre de la persona
- Edad

Descripción de la Estructura Pila

La pila se implementa mediante una clase **Pila**, que contiene:

- Un arreglo de tipo **Persona** llamado **datos**
- Un contador **cantidad** que indica el número de elementos almacenados

La capacidad máxima de la pila es de **100 elementos**.

Operaciones Implementadas

1. Método **agregar(Persona p)**

Permite insertar un elemento en la pila.

- Se almacena el objeto **Persona** en el arreglo
- Se incrementa el contador

Condición:

- Solo se agrega si la pila no está llena

2. Método **quitar()**

Elimina el elemento en la parte superior de la pila.

- Reduce el valor del contador
- El elemento deja de ser accesible

Condición:

- Solo se ejecuta si la pila no está vacía

3. Método **mostrar()**

Imprime los elementos de la pila desde el tope hacia la base.

Formato de salida:

| Nombre, Edad |

Ejemplo:

| Juan, 20 años |

4. Método **estaVacía()**

Verifica si la pila no contiene elementos.

Retorna:

- **true** si está vacía
- **false** en caso contrario

5. Método **estaLlena()**

Verifica si la pila ha alcanzado su capacidad máxima.

Retorna:

- `true` si está llena
- `false` en caso contrario

6. Método `obtenerTamano()`

Devuelve la cantidad actual de elementos en la pila.

Funcionamiento del Programa

Flujo general:

1. Se crea una instancia de la clase `Pila`
2. Se muestra un menú interactivo
3. El usuario selecciona una opción
4. Se ejecuta la operación correspondiente
5. El proceso se repite hasta que el usuario decide salir

Menú del Sistema

El programa incluye las siguientes opciones:

1. Agregar elemento
2. Quitar elemento
3. Mostrar pila
4. Verificar si está vacía
5. Verificar si está llena
6. Mostrar tamaño de la pila
7. Salir

Características de la Implementación

- Tipo de estructura: Lineal
- Implementación: Arreglo estático
- Tipo de dato: Estructura personalizada (`Persona`)
- Capacidad: Fija (100 elementos)
- Acceso: Solo al elemento en el tope

implementación de una Pila Estática mediante Punteros

Concepto General

El presente programa implementa una **estructura de datos tipo pila (stack)** en C++, utilizando memoria dinámica a través de punteros.

Una pila es una estructura lineal que sigue el principio **LIFO (Last In, First Out)**, lo

que significa que el último elemento en entrar es el primero en salir.

En este caso, aunque se utiliza memoria dinámica, la pila sigue siendo **estática**, ya que su tamaño máximo está limitado a 100 elementos.

Descripción de la Implementación

La pila está definida mediante una clase **Pila**, la cual contiene:

- Un puntero **datos** que apunta a un arreglo dinámico de enteros
- Una variable **cantidad** que controla el número de elementos almacenados

El arreglo se reserva dinámicamente en el constructor y se libera en el destructor.

Manejo de Memoria

Constructor

```
Pila::Pila() : cantidad(0) { datos = new int[100]; }
```

- Reserva memoria para 100 elementos
- Inicializa la pila vacía

Destructor

```
Pila::~~Pila() { delete[] datos; }
```

- Libera la memoria utilizada
- Evita fugas de memoria

Operaciones Implementadas

1. Método **agregar(int valor)**

Permite insertar un elemento en la pila.

- Inserta el valor en la posición actual
- Incrementa el contador

Condición:

- Solo se ejecuta si la pila no está llena

2. Método **quitar()**

Elimina el elemento en el tope de la pila.

- Disminuye el contador
- El elemento deja de ser accesible

Condición:

- Solo se ejecuta si la pila no está vacía

3. Método **mostrar()**

Imprime los elementos de la pila desde el tope hacia la base.

Formato:

| valor |

4. Método **estaVacía()**

Verifica si la pila no contiene elementos.

Retorna:

- **true** si está vacía
- **false** en caso contrario

5. Método **estaLlena()**

Verifica si la pila alcanzó su capacidad máxima.

Retorna:

- **true** si está llena
- **false** en caso contrario

6. Método **obtenerTamano()**

Devuelve el número actual de elementos en la pila.

Funcionamiento del Programa

Flujo general:

1. Se crea una instancia de la clase **Pila**
2. Se muestra un menú interactivo
3. El usuario selecciona una opción
4. Se ejecuta la operación correspondiente
5. El proceso se repite hasta que el usuario decide salir

Menú del Sistema

El programa ofrece las siguientes opciones:

1. Agregar elemento

2. Quitar elemento
3. Mostrar pila
4. Verificar si está vacía
5. Verificar si está llena
6. Mostrar tamaño de la pila
7. Salir

Características de la Implementación

- Tipo de estructura: Lineal
- Implementación: Arreglo dinámico mediante puntero
- Capacidad: Fija (100 elementos)
- Acceso: Solo al tope de la pila

pilaLibreria — Pila de enteros con STL

Implementación de una pila de enteros utilizando la biblioteca estándar de C++ (STL).

Objetivo

Proveer una estructura de datos de tipo pila para enteros con las operaciones fundamentales: agregar, quitar, mostrar, verificar estado vacío/lleno y consultar tamaño.

Descripción

El programa presenta un menú interactivo en consola con siete opciones. El usuario puede insertar enteros, extraer el tope de la pila, visualizar todos los elementos, verificar si la pila está vacía o llena, y conocer su tamaño actual. La ejecución continúa en bucle hasta que el usuario elige Salir.

Explicación

- Librería <stack>: provee la clase plantilla `std::stack` que implementa internamente el comportamiento LIFO (último en entrar, primero en salir).
- Librería <cstdio>: usada para entrada/salida con `printf` y `scanf` en lugar de `iostream`.
- Clase Pila: encapsula un miembro privado `std::stack<int>` llamado `datos`. Expone seis métodos públicos.
- `agregar(int valor)`: llama a `datos.push(valor)` para insertar un elemento en el tope.
- `quitar()`: verifica que la pila no esté vacía antes de llamar a `datos.pop()`; muestra un mensaje de error en caso contrario.

- `mostrar()`: copia la pila en una variable temporal para recorrerla sin destruir el contenido original, imprimiendo cada elemento.
- `estaVacia()` / `estaLlena()`: delegan directamente en `datos.empty()`. `estaLlena()` siempre retorna false porque `std::stack` no tiene límite fijo de capacidad.
- `obtenerTamano()`: retorna `datos.size()` convertido implícitamente a int.

Ejemplo de código

```
void Pila::mostrar() const {
    std::stack<int> temp = datos;
    while (!temp.empty()) {
        printf(" | %d |\n", temp.top());
        temp.pop();
    }
}
```

Compilación

```
g++ main.cpp pilaLibreria.cpp -o pilaLibreria
```

pilaLibreriaNTD — Pila de structs con STL

Variante de la pila STL que almacena objetos de un tipo de dato compuesto (struct Persona) en lugar de enteros.

Objetivo

Demostrar que `std::stack` es una plantilla genérica capaz de almacenar cualquier tipo de dato, incluyendo estructuras definidas por el usuario.

Descripción

El funcionamiento del menú es idéntico al programa anterior. La diferencia es que cada elemento insertado es una Persona con nombre (cadena de hasta 50 caracteres) y edad (entero). Al mostrar la pila, se despliegan ambos campos.

Explicación

- `struct Persona`: tipo de dato compuesto con dos campos: `char nombre[50]`

e int edad. Al definirlo fuera de la clase, puede usarse tanto en el .h como en main.

- Clase Pila (con `std::stack<Persona>`): la lógica interna es exactamente la misma que en `pilaLibreria`; únicamente cambia el tipo de plantilla.
- `agregar(Persona p)`: recibe la estructura por valor y la inserta con `datos.push(p)`.
- `mostrar()`: imprime nombre y edad de cada Persona usando `printf` con formato `"%s, %d años"`.
- La entrada de nombre usa `scanf("%s")`, por lo que acepta solo una palabra; para nombres con espacios sería necesario usar `fgets`.

Ejemplo de código

```
void Pila::mostrar() const {
    std::stack<Persona> temp = datos;
    while (!temp.empty()) {
        printf(" | %s, %d años |\n",
            temp.top().nombre, temp.top().edad);
        temp.pop();
    }
}
```

Compilación

```
g++ main.cpp pilalibreria.cpp -o pilaLibreriaNTD
```

Arreglo_colas — Cola de productos con arreglo fijo

Implementación de una cola circular de productos usando un arreglo estático de tamaño fijo.

Objetivo

Implementar una cola con capacidad limitada para almacenar productos (nombre y precio), aplicando la técnica de arreglo circular para reutilizar posiciones liberadas.

Descripción

El menú ofrece las operaciones estándar de cola: encolar, desencolar, mostrar, ver frente, verificar vacío/lleño y consultar tamaño. La capacidad máxima está definida por la constante `MAX`. Al alcanzarse el límite, el programa notifica que la cola está

llena.

Explicación

- Librería <iostream> y <string>: usadas para entrada/salida y para el campo nombre del struct.
- struct Producto: contiene string nombre y float precio.
- Arreglo circular: se usan tres variables enteras (inicio, fin, cantidad) para controlar el estado de la cola. fin avanza con $(fin + 1) \% MAX$ al encolar, e inicio avanza con $(inicio + 1) \% MAX$ al desencolar, lo que permite reutilizar el espacio sin desplazar elementos.
- encolar(Producto p): verifica si la cola está llena; si no, almacena p en arreglo[fin] y actualiza fin y cantidad.
- desencolar(): verifica si está vacía; si no, avanza inicio y decrementa cantidad.
- mostrar(): recorre cantidad elementos desde inicio usando índice $(inicio + i) \% MAX$ para acceder en orden correcto.
- frente(): retorna arreglo[inicio] sin modificar la cola.

Ejemplo de código

```
void Cola::encolar(Producto p) {
    if (estaLlena()) { cout << "Cola llena.\n"; return; }
    arreglo[fin] = p;
    fin = (fin + 1) % MAX;
    cantidad++;
}

void Cola::desencolar() {
    if (estaVacía()) { cout << "Cola vacía.\n"; return; }
    inicio = (inicio + 1) % MAX;
    cantidad--;
}
```

Compilación

```
g++ main.cpp Cola.cpp -o arreglo_colas
```

ND_arreglo_colas — Dos colas con arreglo fijo (NTD)

Extensión del programa anterior que maneja dos colas independientes: una de Productos y otra de Órdenes, ambas con arreglo circular fijo.

Objetivo

Mostrar cómo una misma técnica de implementación (arreglo circular) puede aplicarse a distintos tipos de datos dentro de un mismo programa, usando herencia y clases abstractas.

Descripción

El menú principal permite elegir entre la cola de Productos y la de Órdenes. Cada una tiene su propio submenú con las mismas siete operaciones. Ambas comparten la constante MAX = 5 como límite de capacidad.

Explicación

- struct Producto y struct Orden: Producto contiene nombre (string) y precio (float); Orden contiene id (int) y descripcion (string).
- Clase abstracta ColaAbstracta: define la interfaz de la cola de productos con métodos virtuales puros (encolar, desencolar, mostrar, estaVacia, estaLlena, tamano, frente). Obliga a las clases derivadas a implementar todos los métodos.
- Clase abstracta ColaOrdenAbstracta: interfaz equivalente para Orden.
- ColaProducto y ColaOrden: clases concretas que heredan de sus respectivas abstractas e implementan cada método con la lógica de arreglo circular.
- Funciones manejarProductos() y manejarOrdenes(): encapsulan el submenú de cada tipo de cola, manteniendo el main limpio y modular.

Ejemplo de código

```
class ColaAbstracta {
public:
    virtual void encolar(Producto p) = 0;
    virtual void desencolar() = 0;
    virtual bool estaVacia() const = 0;
    virtual bool estaLlena() const = 0;
    virtual int tamano() const = 0;
    virtual Producto frente() const = 0;
    virtual ~ColaAbstracta() {}
};
```

Compilación

```
g++ main.cpp coland.cpp -o nd_arreglo_colas
```

puntero_colas — Cola de productos con

arreglo dinámico

Implementación de una cola de productos usando un arreglo asignado dinámicamente en memoria con `new[]`.

Objetivo

Implementar la misma funcionalidad de cola que en `Arreglo_colas`, pero administrando la memoria manualmente mediante punteros, lo que permite mayor control sobre el ciclo de vida del arreglo.

Descripción

El comportamiento externo (menú, operaciones, límite MAX) es idéntico al de `Arreglo_colas`. La diferencia es interna: el arreglo se reserva en el constructor con `new[]` y se libera en el destructor con `delete[]`, siguiendo el principio RAII.

Explicación

- Arreglo dinámico con `new[]`: en el constructor, `arreglo = new Producto[MAX]` reserva en el heap el espacio para MAX productos. Esto permite en principio cambiar MAX en tiempo de ejecución si se reestructura el código.
- Destructor `~Cola()`: libera la memoria con `delete[] arreglo`. Sin este paso habría una fuga de memoria al terminar el objeto.
- El resto de la lógica (índices inicio, fin, cantidad y aritmética modular) es idéntica a la implementación con arreglo fijo.
- RAII (Resource Acquisition Is Initialization): patrón de C++ donde los recursos (memoria, archivos, etc.) se adquieren en el constructor y se liberan en el destructor, garantizando limpieza automática.

Ejemplo de código

```
ColaProducto() : inicio(0), fin(0), cantidad(0) {  
    arreglo = new Producto[MAX];  
}  
  
~ColaProducto() {  
    delete[] arreglo;  
}
```

Compilación

```
g++ main.cpp Cola.cpp -o puntero_colas
```

nd_punterocolas — Dos colas con arreglo dinámico (NTD)

Combinación de los enfoques de nd_arreglo_colas y puntero_colas: dos colas (Productos y Órdenes) implementadas con arreglos dinámicos y clases abstractas.

Objetivo

Integrar herencia, polimorfismo y administración dinámica de memoria en un solo programa con dos tipos de datos distintos.

Descripción

El menú principal ofrece elegir entre la cola de Productos y la de Órdenes, cada una con su propio submenú. Internamente, ambas clases concretas asignan su arreglo con `new[]` y lo liberan en el destructor.

Explicación

- ColaAbstracta y ColaOrdenAbstracta: interfaces con métodos virtuales puros, igual que en ND_arreglo_colas.
- ColaProducto hereda de ColaAbstracta; ColaOrden hereda de ColaOrdenAbstracta. Ambas usan arreglo dinámico.
- Destructor virtual en las clases abstractas: declara virtual `~ColaAbstracta() {}` para garantizar que al destruir un objeto por puntero a la clase base se llame al destructor correcto de la clase derivada.
- `mostrar()` en ambas clases: recorre los elementos usando índice modular $(\text{inicio} + i) \% \text{MAX}$, igual que en las versiones con arreglo fijo.
- La separación entre interfaz (abstracta) e implementación (concreta) permite en el futuro sustituir la implementación por otra (e.g. lista enlazada) sin cambiar el código del menú.

Ejemplo de código

```
class ColaOrden : public ColaOrdenAbstracta {
    Orden* arreglo;
    int inicio, fin, cantidad;
public:
    ColaOrden() : inicio(0), fin(0), cantidad(0) {
        arreglo = new Orden[MAX];
    }
    ~ColaOrden() { delete[] arreglo; }
```

```
    // ... resto de métodos  
};
```

Compilación

```
g++ main.cpp cola.cpp -o nd_punterocolas
```

Cola con librería y nuevo tipo de dato

Concepto general

El presente programa implementa una cola (queue) utilizando la librería estándar de C++. La cola sigue el principio FIFO (First In, First Out), es decir, el primer elemento que entra es el primero que sale. En este código maneja un nuevo tipo de dato, normalmente mediante una estructura como Persona o Alumno, la cual guarda algo más que un dato simple.

Descripción del tipo de dato

Se utiliza una estructura personalizada que agrupa varios tipos de datos. Por ejemplo:

```
struct Persona {  
    char nombre[50];  
    int edad;  
    float promedio;  
};
```

Con esto se pueden almacenar datos como nombre, edad y promedio, lo que hace más útil la cola para casos reales. Además de usar otro tipo de datos más que el simple

Descripción del programa

- La cola se maneja con la clase queue de la librería estándar.
- Se almacenan objetos del tipo nuevo de dato dentro de la cola.
- Cada operación respeta el orden de llegada.

Operaciones que se usaron en el programa(cola)

- push(): agrega un elemento al final de la cola.
- pop(): elimina el elemento que está al frente.
- front(): muestra el dato que se va a quitar primero.
- back(): muestra el último elemento agregado.
- empty(): verifica si la cola está vacía.
- size(): devuelve la cantidad de elementos almacenados.

Funcionamiento del programa(Flujo convencional)

- El usuario captura los datos del nuevo tipo de dato.
- El programa agrega cada registro a la cola.
- Después se muestran los elementos en el orden en que deben salir.
- Al final se pueden eliminar uno por uno desde el frente.

Características

- lineal.
- usando librería estándar.
- Tipo de dato: personalizado.
- Acceso: solo al frente y al final.

Lista con arreglo y dato base

Concepto general

El programa hace una lista usando un arreglo dinámico y un dato base, por ejemplo enteros. Una lista es una estructura lineal donde los elementos se almacenan en posiciones consecutivas de memoria y se puede acceder a ellos mediante un índice.

Descripción del programa

La lista se maneja con un arreglo y un contador que indica cuántos elementos hay guardados. El usuario puede agregar, eliminar, buscar y mostrar los elementos de la lista.

Operaciones

- Agregar al final de la lista.
- Insertar en una posición específica.
- Eliminar un elemento por posición o por valor.
- Buscar elementos dentro del arreglo.
- Mostrar toda la lista.
- Verificar si la lista está vacía o llena.

Funcionamiento del programa

- Se define un arreglo con tamaño dinámico, es decir no fijo.
- El usuario introduce los datos base, por ejemplo números enteros.
- Cada dato se guarda en la siguiente posición disponible.
- Cuando se elimina un elemento, los demás se recorren para no dejar huecos.

```
=== Menú de Lista ===
1. Agregar elemento
2. Quitar elemento
3. Mostrar lista
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tamaño de la lista
7. Salir
Elige una opcion: 
```

Características

- lineal.
- arreglo dinamico.
- Tipo de dato: base.
- Capacidad: ilimitada al tamaño del arreglo.

Listas con arreglo y nuevo tipo de dato

Concepto general

El programa usa una lista con arreglo, pero en este caso almacenando un nuevo tipo de dato. La ventaja de este código es que ya no solo se trabaja con números o caracteres, sino con registros completos que representan una entidad, como estudiante.

Descripción del tipo de dato

Se maneja una estructura personalizada que agrupa varios campos. Por ejemplo:

```
struct Estudiante {
    char nombre[50];
    int edad;
    float promedio;
};
```

Cada elemento de la lista representa un registro completo y permite ordenar o mostrar información de manera más clara.

Operaciones

- Agregar un nuevo registro al final.
- Insertar datos en una posición específica.
- Eliminar registros de la lista.
- Buscar un elemento por nombre o por otro campo.
- Mostrar la lista completa con formato legible.

Funcionamiento del programa(flujo)

- Se captura la información de cada registro.
- Los datos se guardan en el arreglo de tipo estructura.
- El contador aumenta conforme se insertan elementos.
- Al mostrar, se recorre el arreglo y se imprime cada campo.

Características

- lineal.
- arreglo dinamico.
- Tipo de dato: nuevo tipo de dato.
- Cada posición guarda un registro completo.

```
Contenido de la lista:
[0]:Codigo 8009
[1]:Codigo 8008
[2]:Codigo 8007
[3]:Codigo 8006
```

```
=== Menú de Lista ===
1. Agregar elemento
2. Quitar elemento
3. Mostrar lista
```

Lista con puntero y dato base

Concepto general

El programa usa una lista utilizando punteros y memoria dinámica. A diferencia del arreglo. En este código se trabajó con un dato base, normalmente enteros.

Descripción de la estructura

La lista está formada por nodos que contienen dos partes: el dato y un puntero al siguiente nodo. Gracias a esto, la memoria se usa solo cuando realmente se necesita.

Ejemplo de nodo

```
struct Nodo {  
    int dato;  
    Nodo* siguiente;  
};
```

```
struct Nodo {  
    int dato;  
    Nodo* siguiente;  
};  
  
void agregarElemento(Nodo*& cabeza) {  
    int valor;  
    cout << "Ingresa un numero entero: ";  
    cin >> valor;  
  
    Nodo* nuevoNodo = new Nodo();  
    nuevoNodo->dato = valor;  
    nuevoNodo->siguiente = nullptr;  
  
    if (cabeza == nullptr) {  
        cabeza = nuevoNodo;  
    } else {  
        Nodo* temp = cabeza;  
        while (temp->siguiente != nullptr) {  
            temp = temp->siguiente;  
        }  
        temp->siguiente = nuevoNodo;  
    }  
    cout << "Elemento agregado.\n";  
}
```

Operaciones

- Insertar al inicio.
- Insertar al final.
- Insertar en una posición específica.
- Eliminar por valor o por posición.
- Buscar un elemento.

- Mostrar toda la lista.

Funcionamiento

- Se crea un puntero para apuntar al primer nodo.
- Cada vez que se agrega un dato, se reserva memoria con new.
- Los nodos se enlazan mediante el puntero siguiente.
- Al eliminar, se reconectan los nodos y se libera la memoria con delete.

Características

- lineal.
- memoria dinámica con punteros.
- Tipo de dato: base.
- Tamaño: variable.

Lista con puntero y nuevo tipo de dato

Concepto general

El programa usa una lista enlazada utilizando punteros, pero ahora con un nuevo tipo de dato. Este enfoque es de los más completos porque combina memoria dinámica con registros complejos, lo cual resulta más útil para manejar información de alumnos, empleados o productos.

Descripción del tipo de dato

Se utiliza una estructura como la siguiente:

```
struct Producto {  
    char nombre[50];  
    float precio;  
    int stock;  
};  
struct Nodo {  
    Producto dato;  
    Nodo* siguiente;  
};
```

Operaciones

- Insertar un nuevo producto al inicio o al final.
- Recorrer la lista para mostrar todos los registros.
- Buscar por nombre o por precio.
- Eliminar un nodo de la lista.
- Actualizar datos de un registro existente.

Funcionamiento

- El usuario captura los datos del tipo de dato personalizado.
- Se crea un nodo dinámico para guardar ese registro.
- El nodo se enlaza con el resto de la lista.
- El recorrido se hace desde el primer nodo hasta que el puntero sea nulo.

Características

- lineal.

- punteros y memoria dinámica.
- Tipo de dato: nuevo tipo de dato.
- La lista crece o disminuye según sea necesario.

Lista con librería y dato base

Concepto general

El programa usa una lista utilizando la librería estándar de C++, en este caso con una estructura como `std::list`. Esta implementación facilita mucho el manejo de inserciones y eliminaciones, ya que no obliga a mover manualmente los elementos como ocurre con un arreglo.

Descripción del programa

La lista almacena un dato base, por ejemplo enteros. La librería se encarga de administrar los nodos internamente, por lo que el programa se enfoca más en usar las operaciones que ya vienen listas para trabajar.

Operaciones

- `push_back()` para agregar al final.
- `push_front()` para agregar al inicio.
- `pop_back()` para eliminar al final.
- `pop_front()` para eliminar al inicio.
- `sort()` para ordenar.
- `reverse()` para invertir el orden.
- `empty()` y `size()` para revisar el estado de la lista.

Funcionamiento

- El usuario ingresa números enteros.
- Cada dato se guarda en la lista de la librería.
- Después se puede recorrer e imprimir en pantalla.
- También se pueden eliminar elementos sin recorrer todo el arreglo manualmente.

Características

- lineal.
- lista de la librería estándar.
- Tipo de dato: base.
- Acceso: secuencial.

Lista con librería y nuevo tipo de dato

Concepto general

El programa usa una lista usando la librería estándar, pero almacenando un nuevo tipo de dato. Esta versión ya se siente más completa porque se trabaja con estructuras que contienen varios atributos, aprovechando las ventajas de la lista de la STL para mover, insertar y eliminar registros con mayor facilidad.

Descripción del tipo de dato

Se define una estructura personalizada, por ejemplo:

```
struct Alumno {  
    char nombre[50];  
    int edad;  
    float promedio;  
};
```

La lista puede almacenar varios alumnos y luego mostrarlos, ordenarlos o eliminarlos según el criterio requerido.

Operaciones

- Agregar registros al inicio o al final.
- Eliminar un alumno específico.
- Recorrer la lista para mostrar los datos.
- Ordenar por promedio o por nombre.
- Verificar el tamaño de la lista.

Funcionamiento

- Se captura cada registro del tipo Alumno.
- El registro se inserta en la lista de la librería.
- Luego se recorren los elementos para mostrarlos en pantalla.
- Si el programa lo permite, también se ordenan antes de imprimirlos.

Características

- lineal.
- lista de la librería estándar.
- Tipo de dato: nuevo tipo de dato.

- Uso sencillo de iteradores o recorridos.

pilaLibrería — Pila de enteros con STL

Implementación de una pila de enteros utilizando la biblioteca estándar de C++ (STL).

Objetivo

Proveer una estructura de datos de tipo pila para enteros con las operaciones fundamentales: agregar, quitar, mostrar, verificar estado vacío/lleno y consultar tamaño.

Descripción

El programa presenta un menú interactivo en consola con siete opciones. El usuario puede insertar enteros, extraer el tope de la pila, visualizar todos los elementos, verificar si la pila está vacía o llena, y conocer su tamaño actual. La ejecución continúa en bucle hasta que el usuario elige Salir.

Explicación

- Librería `<stack>`: provee la clase plantilla `std::stack` que implementa internamente el comportamiento LIFO (último en entrar, primero en salir).
- Librería `<cstdio>`: usada para entrada/salida con `printf` y `scanf` en lugar de `iostream`.
- Clase Pila: encapsula un miembro privado `std::stack<int>` llamado `datos`. Expone seis métodos públicos.
- `agregar(int valor)`: llama a `datos.push(valor)` para insertar un elemento en el tope.
- `quitar()`: verifica que la pila no esté vacía antes de llamar a `datos.pop()`; muestra un mensaje de error en caso contrario.

- `mostrar()`: copia la pila en una variable temporal para recorrerla sin destruir el contenido original, imprimiendo cada elemento.
- `estaVacia()` / `estaLlena()`: delegan directamente en `datos.empty()`. `estaLlena()` siempre retorna false porque `std::stack` no tiene límite fijo de capacidad.
- `obtenerTamano()`: retorna `datos.size()` convertido implícitamente a `int`.

Ejemplo de código

```
void Pila::mostrar() const {
    std::stack<int> temp = datos;

    while (!temp.empty()) {
        printf(" | %d |\n", temp.top());
        temp.pop();
    }
}
```

Compilación

```
g++ main.cpp pilaLibreria.cpp -o pilaLibreria
```

pilaLibreriaNTD — Pila de structs con STL

Variante de la pila STL que almacena objetos de un tipo de dato compuesto (struct Persona) en lugar de enteros.

Objetivo

Demostrar que `std::stack` es una plantilla genérica capaz de almacenar cualquier tipo de dato, incluyendo estructuras definidas por el usuario.

Descripción

El funcionamiento del menú es idéntico al programa anterior. La diferencia es que cada elemento insertado es una `Persona` con nombre (cadena de hasta 50 caracteres) y edad (entero). Al mostrar la pila, se despliegan ambos campos.

Explicación

- `struct Persona`: tipo de dato compuesto con dos campos: `char nombre[50]` e `int edad`. Al definirlo fuera de la clase, puede usarse tanto en el `.h` como en `main`.
- Clase `Pila` (con `std::stack<Persona>`): la lógica interna es exactamente la misma que en `pilaLibreria`; únicamente cambia el tipo de plantilla.
- `agregar(Persona p)`: recibe la estructura por valor y la inserta con `datos.push(p)`.
- `mostrar()`: imprime nombre y edad de cada `Persona` usando `printf` con formato `"%s, %d años"`.
- La entrada de nombre usa `scanf("%s")`, por lo que acepta solo una palabra; para nombres con espacios sería necesario usar `fgets`.

Ejemplo de código

```
void Pila::mostrar() const {  
  
    std::stack<Persona> temp = datos;  
  
    while (!temp.empty()) {  
  
        printf(" | %s, %d años |\n",  
  
            temp.top().nombre, temp.top().edad);  
  
        temp.pop();  
  
    }  
}
```

```
}
```

Compilación

```
g++ main.cpp pilalibreria.cpp -o pilaLibreriaNTD
```

Arreglo_colas — Cola de productos con arreglo fijo

Implementación de una cola circular de productos usando un arreglo estático de tamaño fijo.

Objetivo

Implementar una cola con capacidad limitada para almacenar productos (nombre y precio), aplicando la técnica de arreglo circular para reutilizar posiciones liberadas.

Descripción

El menú ofrece las operaciones estándar de cola: encolar, desencolar, mostrar, ver frente, verificar vacío/lleño y consultar tamaño. La capacidad máxima está definida por la constante MAX. Al alcanzarse el límite, el programa notifica que la cola está llena.

Explicación

- Librería `<iostream>` y `<string>`: usadas para entrada/salida y para el campo nombre del struct.
- struct `Producto`: contiene string nombre y float precio.

- **Arreglo circular:** se usan tres variables enteras (inicio, fin, cantidad) para controlar el estado de la cola. fin avanza con $(fin + 1) \% MAX$ al encolar, e inicio avanza con $(inicio + 1) \% MAX$ al desencolar, lo que permite reutilizar el espacio sin desplazar elementos.
- **encolar(Producto p):** verifica si la cola está llena; si no, almacena p en arreglo[fin] y actualiza fin y cantidad.
- **desencolar():** verifica si está vacía; si no, avanza inicio y decrementa cantidad.
- **mostrar():** recorre cantidad elementos desde inicio usando índice $(inicio + i) \% MAX$ para acceder en orden correcto.
- **frente():** retorna arreglo[inicio] sin modificar la cola.

Ejemplo de código

```
void Cola::encolar(Producto p) {
    if (estaLlena()) { cout << "Cola llena.\n"; return; }

    arreglo[fin] = p;

    fin = (fin + 1) % MAX;

    cantidad++;
}

void Cola::desencolar() {
    if (estaVacía()) { cout << "Cola vacía.\n"; return; }

    inicio = (inicio + 1) % MAX;

    cantidad--;
}
```

Compilación

```
g++ main.cpp Cola.cpp -o arreglo_colas
```

ND_arreglo_colas — Dos colas con arreglo fijo (NTD)

Extensión del programa anterior que maneja dos colas independientes: una de Productos y otra de Órdenes, ambas con arreglo circular fijo.

Objetivo

Mostrar cómo una misma técnica de implementación (arreglo circular) puede aplicarse a distintos tipos de datos dentro de un mismo programa, usando herencia y clases abstractas.

Descripción

El menú principal permite elegir entre la cola de Productos y la de Órdenes. Cada una tiene su propio submenú con las mismas siete operaciones. Ambas comparten la constante `MAX = 5` como límite de capacidad.

Explicación

- `struct Producto` y `struct Orden`: `Producto` contiene `nombre (string)` y `precio (float)`; `Orden` contiene `id (int)` y `descripcion (string)`.
- Clase abstracta `ColaAbstracta`: define la interfaz de la cola de productos con métodos virtuales puros (`encolar`, `desencolar`, `mostrar`, `estaVacia`, `estaLlena`, `tamano`, `frente`). Obliga a las clases derivadas a implementar todos los métodos.
- Clase abstracta `ColaOrdenAbstracta`: interfaz equivalente para `Orden`.
- `ColaProducto` y `ColaOrden`: clases concretas que heredan de sus respectivas abstractas e implementan cada método con la lógica de arreglo circular.

- Funciones `manejarProductos()` y `manejarOrdenes()`: encapsulan el submenú de cada tipo de cola, manteniendo el main limpio y modular.

Ejemplo de código

```
class ColaAbstracta {  
  
public:  
  
    virtual void encolar(Producto p) = 0;  
  
    virtual void desencolar() = 0;  
  
    virtual bool estaVacía() const = 0;  
  
    virtual bool estaLlena() const = 0;  
  
    virtual int tamaño() const = 0;  
  
    virtual Producto frente() const = 0;  
  
    virtual ~ColaAbstracta() {}  
  
};
```

Compilación

```
g++ main.cpp coland.cpp -o nd_arreglo_colas
```

puntero_colas — Cola de productos con arreglo dinámico

Implementación de una cola de productos usando un arreglo asignado dinámicamente en memoria con `new[]`.

Objetivo

Implementar la misma funcionalidad de cola que en `Arreglo_colas`, pero administrando la memoria manualmente mediante punteros, lo que permite mayor control sobre el ciclo de vida del arreglo.

Descripción

El comportamiento externo (menú, operaciones, límite MAX) es idéntico al de Arreglo_colas. La diferencia es interna: el arreglo se reserva en el constructor con `new[]` y se libera en el destructor con `delete[]`, siguiendo el principio RAII.

Explicación

- Arreglo dinámico con `new[]`: en el constructor, `arreglo = new Producto[MAX]` reserva en el heap el espacio para MAX productos. Esto permite en principio cambiar MAX en tiempo de ejecución si se reestructura el código.
- Destructor `~Cola()`: libera la memoria con `delete[] arreglo`. Sin este paso habría una fuga de memoria al terminar el objeto.
- El resto de la lógica (índices inicio, fin, cantidad y aritmética modular) es idéntica a la implementación con arreglo fijo.
- RAI (Resource Acquisition Is Initialization): patrón de C++ donde los recursos (memoria, archivos, etc.) se adquieren en el constructor y se liberan en el destructor, garantizando limpieza automática.

Ejemplo de código

```
ColaProducto() : inicio(0), fin(0), cantidad(0) {  
  
    arreglo = new Producto[MAX];  
  
}  
  
~ColaProducto() {  
  
    delete[] arreglo;  
}
```

```
}
```

Compilación

```
g++ main.cpp Cola.cpp -o puntero_colas
```

nd_punterocolas — Dos colas con arreglo dinámico (NTD)

Combinación de los enfoques de nd_arreglo_colas y puntero_colas: dos colas (Productos y Órdenes) implementadas con arreglos dinámicos y clases abstractas.

Objetivo

Integrar herencia, polimorfismo y administración dinámica de memoria en un solo programa con dos tipos de datos distintos.

Descripción

El menú principal ofrece elegir entre la cola de Productos y la de Órdenes, cada una con su propio submenú. Internamente, ambas clases concretas asignan su arreglo con `new[]` y lo liberan en el destructor.

Explicación

- ColaAbstracta y ColaOrdenAbstracta: interfaces con métodos virtuales puros, igual que en ND_arreglo_colas.

- ColaProducto hereda de ColaAbstracta; ColaOrden hereda de ColaOrdenAbstracta. Ambas usan arreglo dinámico.
- Destructor virtual en las clases abstractas: declara virtual ~ColaAbstracta() {} para garantizar que al destruir un objeto por puntero a la clase base se llame al destructor correcto de la clase derivada.
- mostrar() en ambas clases: recorre los elementos usando índice modular (inicio + i) % MAX, igual que en las versiones con arreglo fijo.
- La separación entre interfaz (abstracta) e implementación (concreta) permite en el futuro sustituir la implementación por otra (e.g. lista enlazada) sin cambiar el código del menú.

Ejemplo de código

```
class ColaOrden : public ColaOrdenAbstracta {
    Orden* arreglo;

    int inicio, fin, cantidad;

public:
    ColaOrden() : inicio(0), fin(0), cantidad(0) {
        arreglo = new Orden[MAX];
    }

    ~ColaOrden() { delete[] arreglo; }

    // ... resto de métodos
};
```

Compilación

```
g++ main.cpp cola.cpp -o nd_punterocolas
```


